COP2034C Module 8 Programming Assignment

Instructions

In this assignment, we will continue building the Palmer Penguins program from Module 7. In that module, we learned how to retrieve individual observations from the Palmer Penguins CSV file, split strings into separate values, convert numeric data, and display formatted information about individual penguins.

In Module 8, we will expand our program to organize and analyze information from multiple penguin observations. Rather than displaying each observation independently, we will collect selected values in a list, perform calculations using those values, and use a dictionary to associate penguin species with their observation counts.

We will practice selected Chapter 8 concepts:

- Creating and modifying lists using append().
- Accessing list elements and using len(), sum(), min(), and max().
- Iterating through lists using a for loop.
- Creating dictionaries containing key-value pairs.
- Accessing dictionary values using keys.

We will continue using the functions, loops, and string-processing techniques introduced in earlier modules, and we will continue using the instructor-provided read_penguin_row() function.

As before, work incrementally. Make one change, run the program, verify the result, and then continue.

1. Start with Your Module 7 Solution

Open the Palmer Penguins notebook you completed for Module 7. Make a copy for Module 8.
A suggested notebook name is:

```
PalmerPenguinsM8.ipynb
```

The Python file you eventually download should be named:

```
PalmerPenguinsM8.py
```

Module 8 should build on your completed Module 7 program rather than starting a new program.

2. Update the ID Header

Update the ID header at the top of the program:

```
# PalmerPenguinsM8.py
# Your Name
# Submission Date
# Analyze Palmer Penguin data using lists and dictionaries
```

Leave a blank line after the header.

3. Retain Your Module 7 Program

Keep the following existing components:

- Constants and functions.
- Parallel lists containing species names and counts.
- Dataset calculations and formatted statistics.
- The instructor-provided read_penguin_row() function.
- The functions that process and display CSV observations.
- Interactive species exploration using the existing while loop.
- The main() function and its call.

We will add a new section to main() that examines a small collection of penguin observations and summarizes their body masses. Continue using the same PalmerPenguins.csv file from Module 7. Remember that Google Colab may require you to upload the file again when starting a new runtime session.

4. Create a List of Penguin Row Numbers

In Module 7, we retrieved two observations by calling process_penguin_row() with individual row numbers. For Module 8, we will use a list to identify several observations we want to analyze. Inside main(), after the existing calls to process_penguin_row(), create the following list:

```
sample_rows = [1, 2, 3, 4, 5]
```

These values identify the first five data rows in the CSV file. They are not body masses or species counts. We will use this list to retrieve observations without writing a separate function call for each row.

5. Create a List to Store Body Masses

Create an empty list named body_masses:

```
body_masses = []
```

This list will store the body mass from each observation. In Module 7, we converted the body mass from a string into an integer. Now we will save several converted values so that we can analyze them together.

Use the append() method to add each body mass to the list as it is retrieved. For example:

```
body_masses.append(body_mass)
```

The append() method adds one element to the end of an existing list.
Do not enter the body masses directly into the list. The values must come from the CSV file.

6. Retrieve and Collect the Body Masses

Create a for loop that processes each row number in sample_rows. For each row:

1. Call read_penguin_row() using the current row number.
2. Use split(",") to separate the CSV row into individual values.
3. Retrieve the body mass using the correct list index.
4. Convert the body mass to an integer.
5. Append the converted value to body_masses.

You may reuse the string-processing statements from Module 7. Use this structure as a guide:

```
for row_number in sample_rows:
    # retrieve the CSV row

    # split the row into individual values

    # convert the body mass to an integer

    # append the body mass to the list
```

Add the appropriate Python statements after each comment. After the loop, display the completed list:

```
print("\nSample Body Masses")
print(body_masses)
```

Verify that the list contains five integer values.

7. Calculate Summary Statistics

Once the list contains the five body masses, use Python's built-in functions to calculate the following:

Statistic	**Python function**
Number of observations	len()
Total body mass	sum()
Smallest body mass	min()
Largest body mass	max()

Calculate the average body mass by dividing the total by the number of observations.
Store each result in an appropriately named variable.

Display the results using f-strings. Format the average body mass to two decimal places.
Your output should include the following:

```
Body Mass Summary
Observations:       5
Total body mass:    ... g
Smallest body mass: ... g
Largest body mass:  ... g
Average body mass:  ... g
```

The ellipses (...) represent values your program must calculate. Do not hard-code the calculated results.

8. Create a Dictionary of Species Counts

Our program already uses two parallel lists to store species names and observation counts:

```
species_names = [    SP_CHINSTRAP,    SP_GENTOO,    SP_ADELIE]
species_counts = [    NUM_CHINSTRAP,    NUM_GENTOO,    NUM_ADELIE]
```

A dictionary provides another way to organize related information. Instead of matching values by their positions in separate lists, a dictionary associates each key with a value.

Inside main(), create a **dictionary** named species_dictionary.

Use the species name constants as keys and the corresponding count constants as values, e.g.,

```
species_dictionary = {
    SP_CHINSTRAP: NUM_CHINSTRAP,
    # add the remaining species here
}
```

Complete the dictionary with all three species. Each species name should be associated with its correct observation count.

9. Access and Display Dictionary Values

Use a for loop to iterate through the dictionary's keys. Inside the loop, retrieve the count associated with each species using dictionary indexing. For example:

```
count = species_dictionary[species]
```

Display the species name and count using an f-string. The output should look like the following:

```
Species Counts Using a Dictionary
Chinstrap: 68
Gentoo: 123
Adelie: 151
```

Notice that now we can retrieve a species count using its name rather than locating its position in a parallel list. Do not remove the original parallel lists; the dictionary is an additional demonstration of a Chapter 8 concept.

10. Organize the New Code in main()

Place the new Module 8 logic after the two existing sample observations and before the interactive species exploration. Your program should now follow this general sequence:

```
main()
    create species lists
    calculate species statistics
    display species statistics

    process first sample observation
    process second sample observation

    create list of sample row numbers
    create empty body mass list

    for each sample row
        retrieve CSV row
        split row into values
        convert body mass
        append body mass to list

    calculate body mass statistics
    display body mass summary

    create species dictionary
    display species counts from dictionary

    explore species using existing while loop
    display final exploration summary
```

Continue using your existing functions where appropriate. Do not remove or redesign working Module 7 functionality.

Expected Output

The program should continue displaying the species statistics and the two sample observations from Module 7. The new Module 8 output should include the following sections:

```
Sample Body Masses
[3750, 3800, 3250, 3450, 3650]

Body Mass Summary
Observations:       5
Total body mass:     17900 g
Smallest body mass:  3250 g
Largest body mass:   3800 g
Average body mass:   3580.00 g

Species Counts Using a Dictionary
Chinstrap: 68
Gentoo: 123
Adelie: 151

Explore the Palmer Penguins
Enter Chinstrap, Gentoo, Adelie, or Done.
```

The actual body mass values and calculated results depend on the contents of the provided CSV file. Minor whitespace differences are acceptable as long as the output is readable and includes the required information.

Test Your Program

Before submitting, verify that:

- The original Module 7 program still runs correctly.
- Two individual penguin observations are displayed.
- The sample_rows list contains five row numbers.
- A for loop retrieves five observations from the CSV file.
- The append() method stores five integer body masses.
- The total, minimum, maximum, and average body masses are calculated correctly.
- The species dictionary contains all three species and their correct counts.
- A for loop displays the dictionary contents.
- The interactive species exploration still works, and Done ends the loop.

Data Science Connection

In Module 7, we learned how to retrieve individual observations from a dataset and convert their values into usable Python data types. In Module 8, we take the next step by collecting multiple observations and performing basic statistical analysis.

Lists are useful in data science because they allow us to store collections of related measurements. For example, a list of penguin body masses can be used to calculate the average, identify the smallest and largest measurements, and compare observations. Instead of writing separate calculations for each penguin, we can use a loop to collect the data and built-in Python functions to summarize it.

Dictionaries provide another important way to organize data. By associating species names with observation counts, we can retrieve information using meaningful keys rather than numeric indexes. This is useful when working with categories, such as species, geographic locations, or experimental groups.

Our program now demonstrates a more complete data-science workflow:

- Retrieve data from the CSV file
- Parse and convert individual values
- Collect observations in a list
- Calculate summary statistics
- Organize and display results

Although we are working with only five observations, these same principles apply to much larger datasets. In later modules, we will expand our ability to process files and analyze larger collections of data.

Readability

Continue following the readability expectations from previous modules:

- Meaningful variable and function names.
- Consistent four-space indentation.
- Reasonable line lengths and logical blank lines.
- Comments identifying major sections and explaining important operations.

Useful new comments might include:

```
# create a list of sample row numbers
# collect body masses from the CSV data
# calculate summary statistics
# associate species names with observation counts
```

Avoid duplicating code unnecessarily, and preserve the organization established in earlier modules.

Screenshot and Submission

Follow the same submission process used for Module 7. Run the completed program in Google Colab. For the screenshot, show enough of one execution to demonstrate the new body mass summary, the species dictionary output, and the existing interactive species exploration.

Suggested filenames:

```
PalmerPenguinsM8.py
PalmerPenguinsM8.png
```

Upload both files to your forked GitHub repository and submit the repository URL to Canvas. Do not submit the .ipynb notebook.